

marp: true
theme: default
paginate: true



SaglamOl

A Microservices E-Health Insurance Platform

Buy plans · Submit claims · AI-driven fraud detection & auto-decisions

Team: Nicat Nasirli · Ağahüseyn Quluzadə · Ramin Aliyev · Sadiq Səfərov

The Problem

Traditional health-insurance claim handling is:

- 🐢 **Slow** — every claim waits in a manual review queue
- 🧑 **Labor-heavy** — staff review *all* claims, even obvious ones
- 🤪 **Fraud-prone** — inflated or duplicate claims are hard to catch at scale
- 🔌 **Monolithic** — one big system; hard to scale or change one part

Our goal

An online platform where customers self-serve (buy plans, file claims),
AI scores fraud and auto-decides routine claims, and staff focus only on the uncertain ones — built as **independent, scalable microservices**.

What We Built

SaglamOI — a B2C **single-insurer** platform (the platform *is* the insurer).

Deliberately scoped as an MVP: no multi-tenant insurers, hospitals, or doctors.

The claim lifecycle, end to end

1. **Register / log in** → JWT-secured account
2. **Browse & buy a plan** → payment runs automatically → policy goes **ACTIVE**
3. **Submit a claim** → **AI scores fraud risk** → auto-decision:
 - low risk → **APPROVED** (payout paid automatically)
 - medium → **UNDER REVIEW** (staff queue)
 - high → **REJECTED**
4. **Track** policies, claims, payments & notifications · ask the **AI assistant**

Three roles

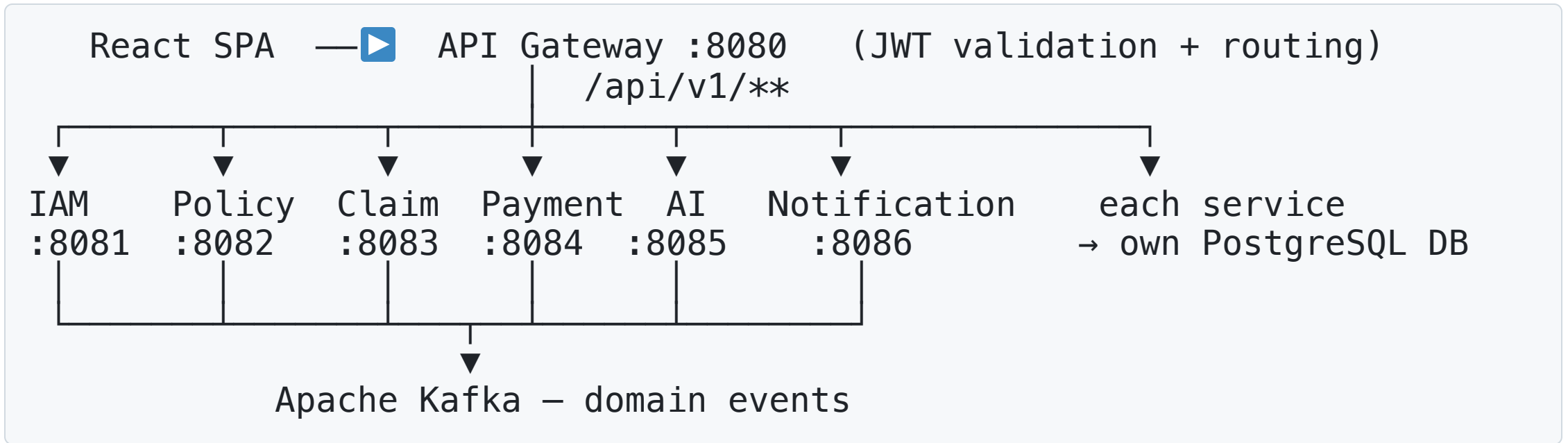
Key Features

 Customer	 Staff	 Admin
Register / login	Review queue (uncertain claims)	Create & delete plans
Browse plan details	Approve / reject claims	View all users / policies
Buy & cancel a policy	Inspect AI fraud assessment	Search users
Submit claims + evidence	Member search	Change roles · suspend users
Track claims & payments		View all claims / payments
Real-time notifications		
AI assistant (chatbot)		

Tech Stack

Layer	Technology
Language / runtime	Java 21
Framework	Spring Boot 3.3.5 (Web, Data JPA, Security, Validation)
Messaging	Apache Kafka — event-driven backbone
Database	PostgreSQL 16 — one database per service
Auth	JWT (jjwt) + Spring Security
AI	OpenRouter LLM (<code>google/gemini-2.5-flash</code>) via WebClient
Mapping / build	MapStruct · Gradle · Liquibase (schema migrations)
Frontend	React 18 + TypeScript + Vite
Infra / tooling	Docker Compose · Swagger / OpenAPI · Kafka UI

Architecture Overview



- **Gateway** — single entry point; validates the JWT, routes to services
- **6 services** — each owns its **own database** (no shared tables)
- **Kafka** — services communicate by **events**, never direct service-to-service calls
- **Kafka UI + Swagger** — inspect events and APIs

Why Microservices + Event-Driven?

Why split into services?

- **Independent scaling & deployment** — change AI/claims without touching the rest
- **Clear ownership** — each service owns its data and its domain
- **Fault isolation** — one service failing doesn't take down the others

Why Kafka events instead of direct calls?

- The real flows are **naturally asynchronous** (payment ~2s, AI scoring)
- **Decoupling** — the claim service doesn't need to know payment or AI exist
- **Resilience** — if a consumer is down, events wait and are processed later

The honest trade-off

Distributed systems add complexity: **eventual consistency** and the need for

The Services at a Glance

Service	Responsibility	Events (→ produces / ← consumes)
Gateway	Single entry; JWT validation + routing	—
IAM	Users, auth (JWT), roles	→ <code>user.registered</code>
Policy	Plans & policies	→ <code>policy.created</code> · ← payment
Payment	Premium charges & payouts (mock)	→ <code>payment.completed/failed</code> · ← policy, claim
Claim	Claims, evidence, auto-decision	→ <code>claim.submitted/decision</code> · ← <code>fraud.detected</code>
AI	Fraud scoring + chatbot	→ <code>fraud.detected</code> · ← <code>claim.submitted</code> , <code>policy.created</code>

Policy Service — Buying a Policy

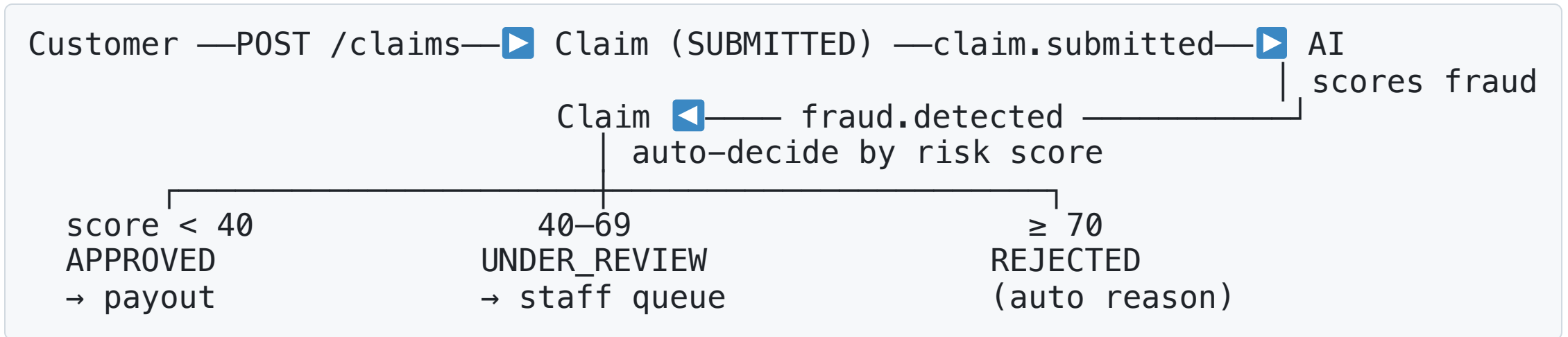
Owns the plan catalog + customer policies. Showcases the payment saga.

```
Customer —POST /policies—▶ Policy: create policy (PENDING)
                             | publish policy.created
                             ▼
                             Payment: charge premium (mock ~2s)
                             | publish payment.completed
                             ▼
                             Policy: policy → ACTIVE (start/end dates)
                             Notification: "policy activated" email
```

- HTTP response returns **PENDING** immediately → frontend re-queries → **ACTIVE**
- **One active policy rule** — purchasing a second is rejected (400)
- Payment **fails** → `payment.failed` → policy auto-**CANCELLED** + notification
- Event-handling methods are **@Transactional** — safe on redelivery

Claim Service — Submit & Auto-Decide

Owns claims + evidence. The heart of the workflow.



- **Auto-decision** removes humans from the routine path
- Staff review only **UNDER_REVIEW** claims (PUT /claims/{id}/review)
- APPROVED → claim.decision → **payout** + notification
- Evidence files uploaded via multipart and linked to the claim

AI — Fraud Detection

A **hybrid** score: deterministic rules + an LLM second opinion.

1. Rule score (fast, explainable)

- Claim amount vs the policy's **coverage** → **plan-based, not hardcoded**
 - **> 50%** of coverage → +40 · **> 100%** → +20
- Repeat high-risk user → +20

2. LLM assessment (only if rule score ≥ 40)

- Sends claim context to the LLM → returns a score + explanation + flags

3. Final score = **rule $\times$ 0.4 + ai $\times$ 0.6**

→ published as **fraud.detected** → drives the claim's auto-decision

*The AI learns each policy's coverage from a **policy.created** read-model —*

AI — Customer Assistant (Chatbot)

A support chatbot for insurance questions, powered by the same LLM.

-  **Conversational** — remembers context within a session (each message replays the session history to the model)
-  **Scoped** — system prompt restricts it to insurance topics (plans, policies, claims, coverage); politely declines off-topic asks
-  **Persisted** — every message stored (`chat_messages`), retrievable by session
-  Same OpenRouter LLM as fraud detection, isolated in the **AI service**

```
POST /api/v1/ai/chatbot { sessionId?, message } → { sessionId, reply }
```

Reliability & Correctness

Event-driven systems can double-act or diverge on redelivery. We handled it.

Idempotent consumers

- **Payment** — a *mock processor* (auto-completes, no real gateway), but it still **skips if a payment already exists** for the reference → the same guard that would prevent a double charge / double payout against a real provider
- **Policy / Claim** — status guards (`if not PENDING/SUBMITTED → return`)
- **Notification** — `correlationId` dedup → no duplicate emails

Atomic save-then-publish

- State-changing handlers are `@Transactional` → if the publish fails, the DB write rolls back and redelivery re-runs cleanly (no state/event divergence)

Security

-  **JWT authentication** — issued by IAM on login; validated at the gateway *and* by each service; signed with a shared secret (HS384)
-  **Role-based access** — `@PreAuthorize` enforces CUSTOMER / STAFF / ADMIN on every protected endpoint
-  **Gateway as the single entry** — only `/api/v1/auth/**` and `GET /plans` are public; everything else requires a valid token
-  **Passwords** — BCrypt-hashed, never stored or returned in plaintext
-  **Secrets via environment** — JWT secret & API keys injected from `.env` (git-ignored); compose **fails fast** if the secret is unset

Honest note: this is dev-grade config — production would add a secrets manager and key rotation.

Tooling & Testing

Testing — 217 tests across services, all green

- **Unit tests** (JUnit 5 · Mockito · AssertJ) — service logic: decision thresholds, fraud scoring, ownership/validation
- **Integration tests** (`@SpringBootTest` · Testcontainers / Embedded Kafka) — full event chains (e.g. `policy.created → payment → activation`)

Tooling & developer experience

- **Swagger / OpenAPI** per service — interactive API docs + "Authorize" with JWT
- **Kafka UI** (`:8090`) — browse topics, messages & consumer-group lag
- **Postman collection** — auto-chains tokens & ids through the whole flow
- **Docker Compose** — one command brings up the whole stack

Live Demo

A full customer journey, end to end:

1. **Register** a customer → welcome email arrives (*show inbox / logs*)
2. **Browse plans** → **buy a policy** → re-query → status **ACTIVE** (*async activation*)
3. **Submit a claim** → watch the **AI auto-decision** (approved / review / rejected)
4. **Kafka UI** (:8090) → show the events flowing across topics
5. **Staff** logs in → review an **UNDER_REVIEW** claim → approve → payout
6. **AI assistant** → ask an insurance question, then an off-topic one (*it declines*)
7. **Admin** → create a plan · view users / policies

Fallback: screenshots of each step in case of live issues.

Challenges, Decisions & Next Steps

Challenges we solved

- **Redelivery** → **double actions** → idempotent consumers + `@Transactional`
- **Async UX** (payment/activation not instant) → **PENDING** + re-query pattern
- **AI for fraud** → hybrid rule + LLM, **plan-based** thresholds
- **Cross-service contracts** → shared `infra` library (events / DTOs / enums)

Deliberate scope cuts (MVP)

- Mock payment (no real gateway) · single-insurer · no Kafka dead-letter topic yet

Next steps

- Dead-letter topics + retry/backoff · real payment integration
- Observability (tracing / metrics) · frontend tests · production secrets manager



Thank You

Questions?

SaglamOI — a microservices e-health insurance platform

Team: Nicat Nasirli · Ağahüseyn Quluzadə · Ramin Aliyev · Sadiq Səfərov